METODE REGULA FALSI

Komputasi dan Pemrograman Lanjutan

Kelompok 2

Anggota Kelompok

1. Asrini (230101500023)
2. Diana Adelia Putri (230101501006)
3. Arinda Aulya (230101501007)

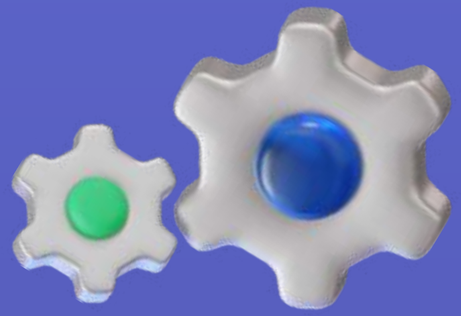
Nama Metode:

Metode Regula Falsi

$$c = \frac{a \times f(b) - b \times f(a)}{f(b) - f(a)}$$

Nama Class:

- class Bisection
- class RegulaFalsi
- class ErrorCalculator



Struktur Atribut, Method dan Desain OOP

Class: RegulaFalsi

- Constructor: __init__
- Attributes: func, a, b, tol, max_iter, iteration
- Methods: hitung_titik(), solve(), get_iterations()

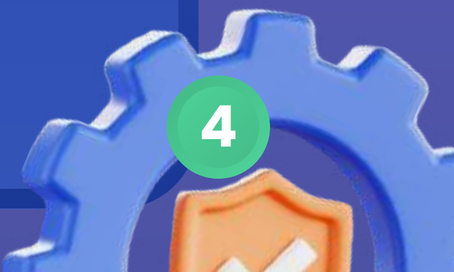
Algoritma Metode

1. Membaca fungsi dari user
Program meminta pengguna memasukkan fungsi f(x) dalam bentuk string.

```
fungsi_input = input("Masukkan fungsi f(x): ")
```

2. Membaca parameter
Program membaca nilai: batas bawah a, batas atas b, toleransi, maksimum iterasi

```
a = float(input("Masukkan batas bawah (a): "))
b = float(input("Masukkan batas atas (b): "))
tol = float(input("Masukkan toleransi: "))
max_iter = int(input("Masukkan maksimum iterasi: "))
```



Algoritma Metode

3. Mendefinisikan fungsi
Input string diubah menjadi fungsi matematika yang dapat dihitung.

```
def f(x):
    return eval(fungsi_input)
```

4. Validasi fungsi
a. Validasi bentuk fungsi
Program mengecek apakah fungsi dapat dijalankan.

```
try:
    f(1)
except:
```

Algoritma Metode

b. Validasi interval
Program memastikan bahwa f(a) dan f(b) memiliki tanda berbeda.

```
if f(a) * f(b) > 0:
```

Jika tidak, program dihentikan.

5. Menjalankan metode Regula Falsi
a. Menghitung akar
Program membuat objek Regula Falsi dan menjalankan metode solve().

```
rf = RegulaFalsi(f, a, b, tol, max_iter)
akar_rf = rf.solve()
```

b. Menyimpan data iterasi
Data setiap iterasi otomatis disimpan dalam atribut iterations.

Algoritma Metode

6. Menampilkan hasil Regula Falsi

a. Menampilkan hasil Regula Falsi

```
print("Akar:", akar_rf)
```

b. Menampilkan tabel iterasi

```
for data in rf.get_iterations():
```

7. Menjalankan metode Bisection

a. Menghitung akar

```
bs = Bisection(f, a, b, tol, max_iter)
akar_bs = bs.solve()
```

b. Menyimpan data iterasi

Disimpan dalam atribut iterations pada objek Bisection.

Algoritma Metode

8. Menampilkan hasil Bisection
a. Menampilkan akar
b. Menampilkan tabel iterasi
Prosesnya sama seperti Regula Falsi

9. Mengambil data error
Program mengambil nilai error dari setiap iterasi kedua metode.

```
rf_errors = [data['error'] for data in rf.get_iterations()]
bs_errors = [data['error'] for data in bs.get_iterations()]
```

Algoritma Metode

10. Membuat grafik konvergensi
Program memplot error terhadap iterasi untuk membandingkan kedua metode.

```
plt.plot(...)
```

11. Menampilkan grafik
Grafik ditampilkan ke layar.

```
plt.show()
```

Cuplikan Kode Program

Calculate
Eror

Regula
Falsi

Main.ipynb

Penggunaan Library

x**3-x-2

Masukkan fungsi f(x): (Press 'Enter' to confirm or 'Escape' to cancel)

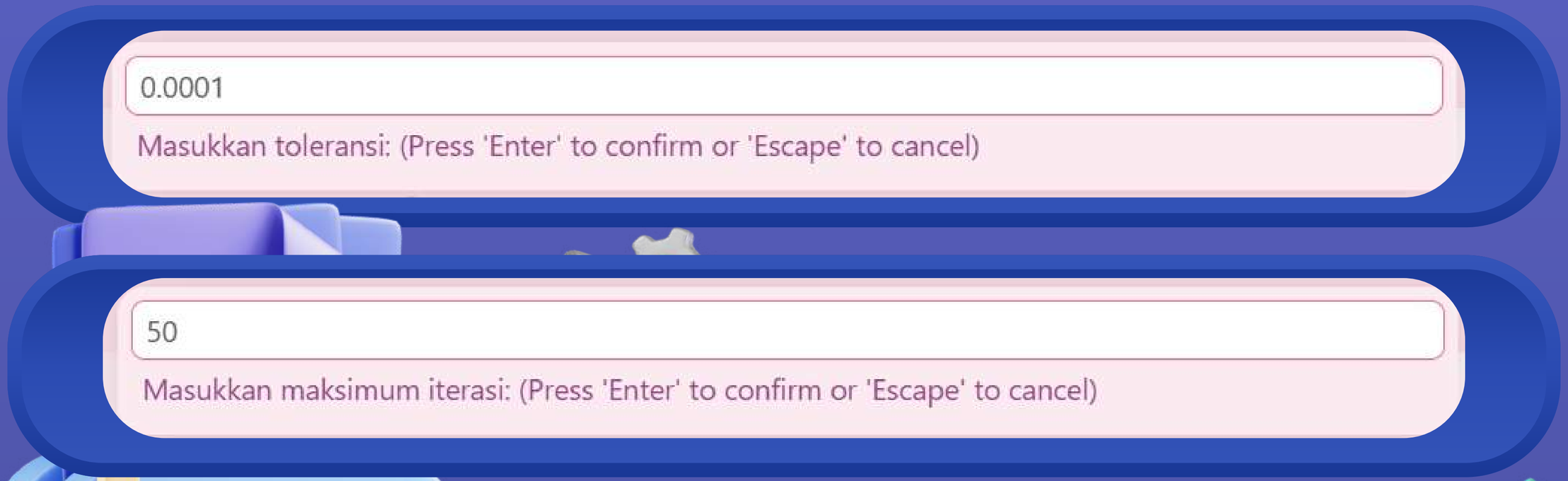
1

Masukkan batas bawah (a): (Press 'Enter' to confirm or 'Escape' to cancel)

2

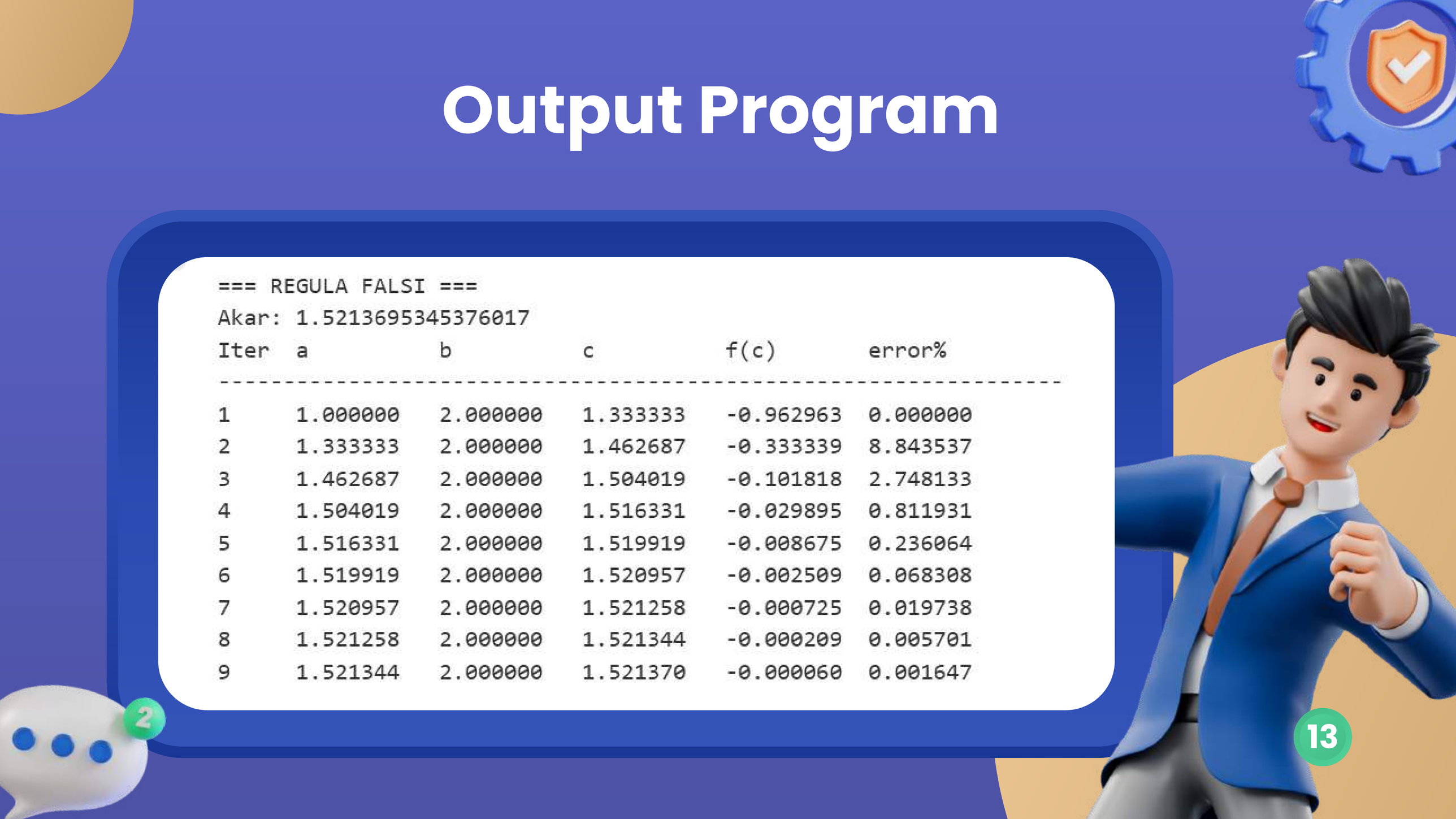
Masukkan batas atas (b): (Press 'Enter' to confirm or 'Escape' to cancel)

Penggunaan Library



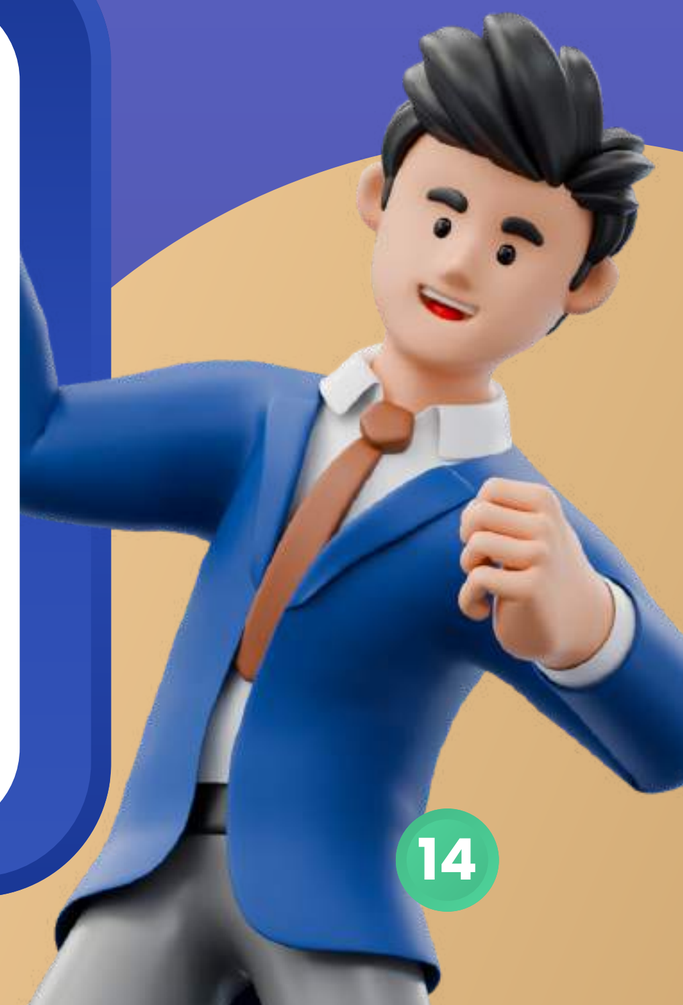
Output Program

```
=== REGULA FALSI ===
Akar: 1.5213695345376017
Iter  a          b          c          f(c)       error%
-------------------------------------------------------------------
1     1.000000   2.000000   1.333333   -0.962963  0.000000
2     1.333333   2.000000   1.462687   -0.333339  8.843537
3     1.462687   2.000000   1.504019   -0.101818  2.748133
4     1.504019   2.000000   1.516331   -0.029895  0.811931
5     1.516331   2.000000   1.519919   -0.008675  0.236064
6     1.519919   2.000000   1.520957   -0.002509  0.068308
7     1.520957   2.000000   1.521258   -0.000725  0.019738
8     1.521258   2.000000   1.521344   -0.000209  0.005701
9     1.521344   2.000000   1.521370   -0.000060  0.001647
```

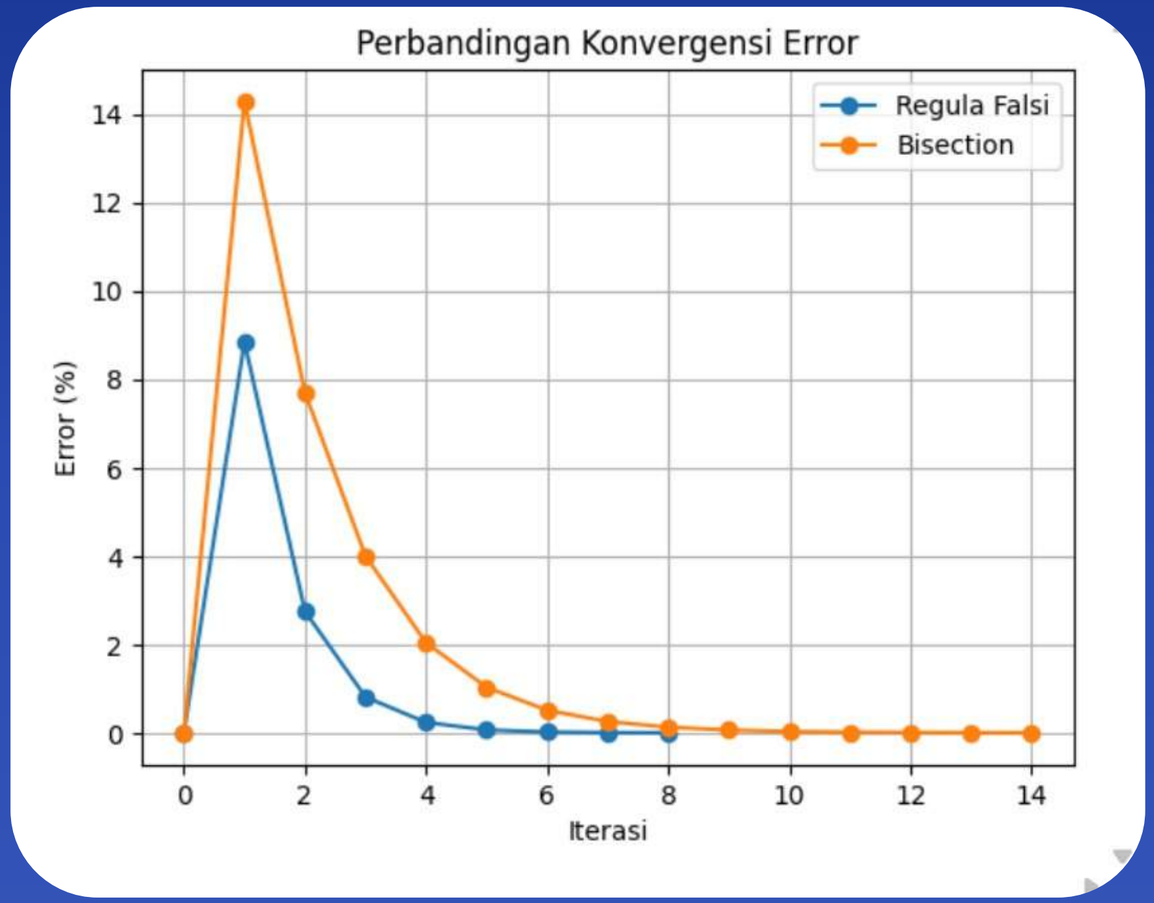


Output Program

```
=== BISECTION ===
Akar: 1.521392822265625
Iter  a          b          c          f(c)       error%
-------------------------------------------------------------------
1     1.000000   2.000000   1.500000   -0.125000  0.000000
2     1.500000   2.000000   1.750000   1.609375   14.285714
3     1.500000   1.750000   1.625000   0.666016   7.692308
4     1.500000   1.625000   1.562500   0.252197   4.000000
5     1.500000   1.562500   1.531250   0.059113   2.040816
6     1.500000   1.531250   1.515625   -0.034054  1.030928
...
12    1.520996   1.521484   1.521240   -0.000829  0.016049
13    1.521240   1.521484   1.521362   -0.000103  0.008024
14    1.521362   1.521484   1.521423   0.000259   0.004012
15    1.521362   1.521423   1.521393   0.000078   0.002006
```



Plot Visualisasi (Grafik)



Kesimpulan

metode Regula Falsi digunakan untuk menentukan akar persamaan nonlinier dengan pendekatan interpolasi linear pada dua titik yang mengurung akar dan diperbarui secara iteratif hingga memenuhi toleransi. Dibandingkan metode Bisection yang membagi interval menjadi dua bagian sama besar, Regula Falsi umumnya lebih cepat konvergen, namun Bisection lebih stabil karena interval selalu dipersempit secara konsisten meskipun konvergensinya lebih lambat.

TERIMA KASIH